

A developer who digs into root causes and enjoys solving difficult problems

The problems I encountered, the experiments and results, and the decisions that shaped each implementation.

Jaehyeong Kim

AI Application Developer · Backend Engineer
kkuldangi2@gmail.com · martinel2.github.io

I owned the whole AI side of an LLM workspace, designing and building its RAG, agent and evaluation layers.

I weigh accuracy, cost and response time against each other, confirm the trade-off with an evaluation, and then choose the implementation that fits the service.

RAG evidence selection: coverage 66.25% → 91.25%, false evidence 20 → 0

Built a golden set and ran a blinded offline comparison with a matched candidate pool and two independent judge models, then replaced the evidence-selection stage. Tuning candidate count and parallel batches cut median selection time from 9.090s to 2.251s.

Agent execution environment: user-defined skills behind approval, edit quality 94/114 → 104/114

Split responsibility so code enforces permissions and prompt-injection checks, the model drafts instructions and edits, and the user publishes and approves. Nothing changes in a document before the user approves it.

Data pipelines: PDF conversion pass rate 45.17% → 89.89%, drug conversion \$200 estimated → \$11.18 spent

Separated processing paths for body text, tables and equations so documents become usable for retrieval and answers, and processed about 44,000 drug records within budget through deduplication, ingredient-level backfill and batch conversion.

Contents

FRUITION

01 RAG evidence coverage: 66.25% → 91.25% p.6

02 Agent execution environment: authoring, validation and approval p.13

03 PDF conversion: pass rate 45.17% → 89.89% p.18

04 Parallel analysis: 73.8% less processing time p.22

PILLTIP

05 Drug data conversion: \$200 estimated → \$11.18 spent p.27

06 Medication chatbot: health profiles kept internal p.32

MORE

07 Beyond project development p.35

08 Project reflections p.36

Fruition

An AI workspace that organizes uploaded documents into an LLM-generated Wiki, answers questions using source evidence through RAG, and edits documents in natural language. It is currently deployed on AWS EKS and undergoing QA.

How it started

Started when a teammate raised a pain point with managing documents, which we refined through design thinking. The goal was to fix the frustration of saving many documents but failing to find them when needed.

GitHub · AI repository ↗

Team and my role

MY ROLE

AI

All AI features: PDF conversion, Wiki generation, RAG retrieval and grounded answers, the document-editing Agent

01 Lead for all AI features

Led design, development and evaluation across PDF conversion, Wiki generation and organization, retrieval and grounded answers, and the document-editing Agent.

02 MSA design contribution

Proposed the initial architecture, then finalized service boundaries, data ownership and communication with teammates and mentors.

Backend, 1

The entire Spring-based backend

Frontend & DevOps, 1

The entire frontend and MSA-based deployment on AWS

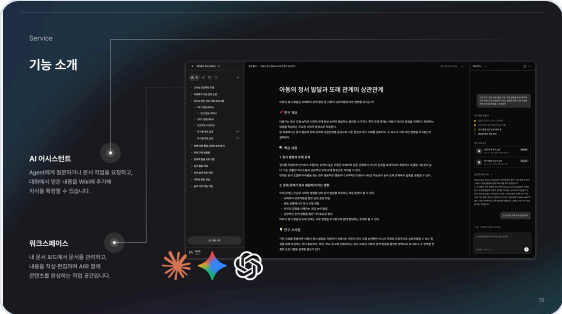
Design

External designer

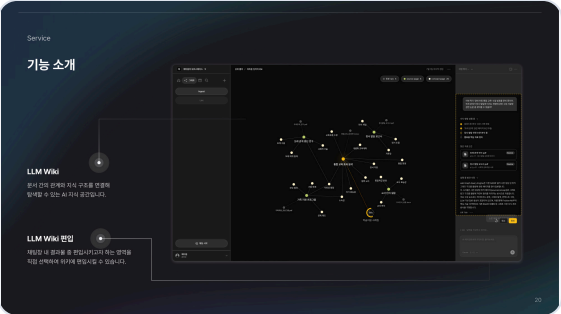
Mentors, 3

Gen AI quality and evaluation / backend / milestones, MSA design and documentation

Features and service screens



Document editing and AI assistant
I led the AI features that interpret requests and generate and validate edits. Teammates implemented the editor and assistant UI.



Wiki generation and retrieval
I led the AI features that organize and connect document knowledge and find evidence for questions. Teammates implemented the graph and search UI.

Architecture

시스템 아키텍처

브라우저는 Spring Boot API를 통해 데이터와 AI Service을 사용합니다.

기술	구현
Next.js	문서 · Wiki · Chat · Agent UI
Spring Boot	인증 · 권한 · 워크스페이스 · 상태
FastAPI	Ingest · Query · Lint · 문서 편집 Agent
PostgreSQL	관계 · 버전 · 작업 상태
MinIO	원본 · Markdown · artifact

Service responsibilities

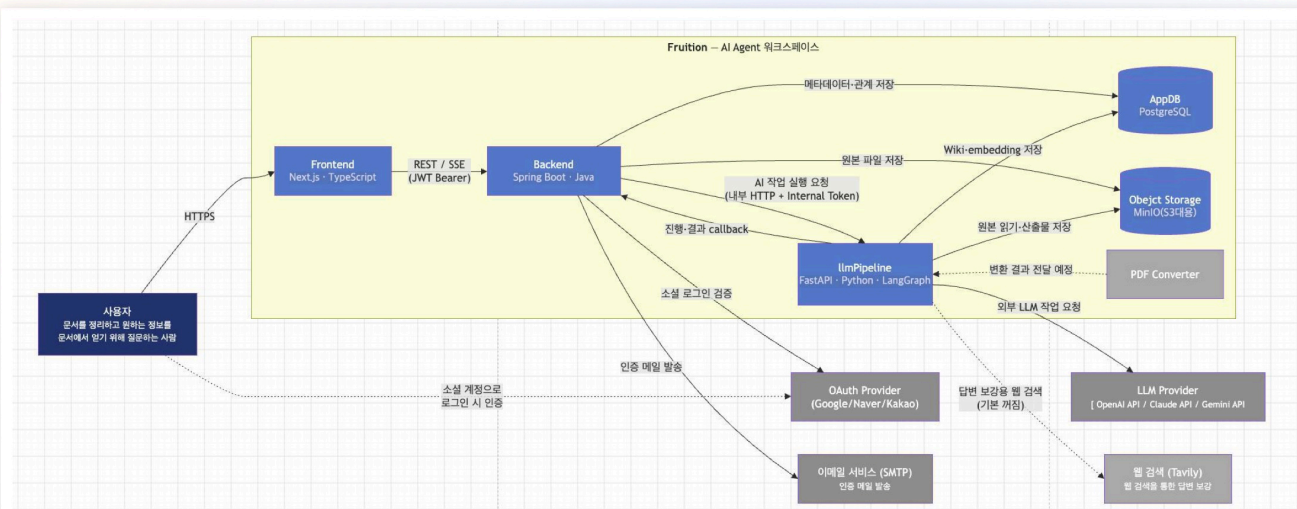
I led all AI features; teammates owned frontend and backend. We finalized MSA with mentors based on my initial design.

System architecture

| C1 전체 시스템 아키텍처 간략화



| C2 Fruition 시스템 일부 구체화



the overall system (top) and Fruition's internal components (bottom). I owned the AI features centred on llmPipeline (FastAPI · Python · LangGraph); teammates owned the frontend, backend and AWS deployment.

RAG evidence coverage: 66.25% → 91.25%, with 75% less time after tuning

Jev is a decision-only model that must pick from a fixed set of options. Applied first to request routing, it scored 81/98 against 77/98 for the existing JSON approach and 63/98 for GPT-5-nano on the same options, with a shorter decision time. I then applied it to RAG evidence selection, increasing coverage from 66.25% to 91.25%, and reduced its added latency by tuning candidates and parallel batches.

2026.09

Jev 1.13.0

BGE-M3

BM25

RAG

Quality evaluation

Background

To choose appropriate tasks and answer evidence more reliably, I evaluated Jev first for routing and then expanded it to RAG.

Evidence coverage
(LLM evaluation on
golden set)

**66.25% →
91.25%**

Jev evidence-selection
time

**Median
9.090s →
2.251s**

False evidence returns
on 20 unanswerable
questions

20 → 0

Experiments and improvements

1 Routing comparison and the decision to try RAG

Problem

Some questions were misclassified as document edits, leading to the wrong action. Task types and additional conditions needed to be selected more accurately and quickly.

Experiment and result

Applied Jev, a choice-based decision model released on September 14, to routing questions, document edits, document creation and folder organization.

Across 98 requests, correct task types and additional conditions totaled 77 for the existing JSON approach, 63 for GPT-5-nano given the same options, and 81 for Jev. Median decision time improved from 8.967s to 0.658s.

I measured API cost on the same 100 questions: about \$0.0408 for Jev against about \$0.1027 for the GPT approach given the same options.

Decision

Jev showed promise on accuracy, speed and cost alike. On that basis I decided to try it on evidence selection, which previous retrieval changes had not solved.

Experiments and improvements (continued)

2 Full-corpus inspection: evidence coverage on all 90 answerable questions

Problem

Using the original question for retrieval and increasing semantic-score weight raised top-1 correct retrieval from 50 to 57 out of 110 questions. However, the experiment had some bias and did not establish reliable evidence selection on new questions.

Experiment and result

Tested whether Jev could find evidence missed by retrieval. It inspected all 2,020 evidence units in a test Wiki without retrieval filtering and returned at most 8, matching the existing return limit.

In a semantic evaluation allowing paraphrases, coverage on 90 answerable questions rose from 24/90 (26.7%) to 90/90 (100%). It returned no evidence for all 10 unanswerable questions. Median processing time was 1.511s for the original approach and 5.338s for Jev. Jev processed 26 batches of 80, with up to 4 in parallel.

Full-corpus inspection took 2,600 calls for 100 questions, and the API reported roughly 124.5M input and 6.47M output tokens. I did not confirm the billed amount.

To understand the size of the gap I traced where the existing retrieval dropped the answer. The page holding the reference sentence was in the initial candidate pool for all 90 questions, but 76 fell out at the step that limits inspection to the top 8 pages, 8 at the threshold of 85% of the best in-page score, and the remaining 2 at the final 8-evidence cap and the related-page lookup.

Decision

The existing retrieval was discarding the answer before selection even began, so the gap cannot be read as a difference in selection ability over the same candidates. Full-corpus access found the evidence, but inspecting everything per question would grow calls as documents grew. Rather than set the deployment scope from this 90-question wiki alone, I decided to narrow candidates through retrieval first and let Jev pick the final evidence.

Experiments and improvements (continued)

3 Controlled comparison on 500 candidates: 87.50% coverage

Problem

The full-corpus test needed 26 batches per question. Questions were concentrated in certain technical documents and included document titles. The original search inspected only some pages while Jev saw all evidence, confounding judgment quality with access scope. Both candidate count and comparison conditions needed control.

Experiment and result

Created 100 new questions from 9 papers without reusing earlier questions: 63 answerable from one paper, 17 requiring two papers, and 20 unanswerable. Questions, reference answers and required facts were fixed before runs.

After deduplication, BGE-M3 and BM25 selected up to 500 candidates once; both methods received identical text and metadata. The original selector could inspect every candidate page, and both methods returned at most 8 untruncated evidence passages. Reference answers were excluded from retrieval and Jev inputs. GPT and Gemini graded independently without seeing method names or competing outputs, and run order alternated.

GPT-graded coverage rose from 53/80 (66.25%) to 70/80 (87.50%); Gemini also showed improvement, from 52/80 to 74/80. False evidence returns on unanswerable questions fell from 20 to 1. Median time including candidate preparation was 0.431s for the original selector and 9.090s for Jev, which processed batches sequentially.

Decision

Jev selected better evidence even with matched candidates and return conditions, so I kept retrieval for candidate collection and Jev for final selection. Results on public papers and self-authored questions were not treated as general accuracy. Next I would tune candidate count and batch processing to reduce sequential waiting.

Experiments and improvements (continued)

4 300 candidates and 4 parallel batches to reduce latency

Problem

Jev improved evidence selection, but sequential processing of up to 500 candidates raised median preparation-and-selection time to 9.090s versus 0.431s previously. I needed to reduce waiting without losing accuracy.

Experiment and result

Reduced the candidate pool to the top 300 and ran up to 4 batches in parallel. Time fell to 2.251s, while questions with all required evidence increased from 70 to 73.

Decision

At 2.251s, the tuned approach was still slower than the original 0.431s. I considered fewer candidates and more parallel batches, but rejected further candidate reduction because some required evidence ranked below 250.

I adopted Jev despite the added latency because accuracy was central to the service.

Experiments and improvements (continued)

5 Comparing candidate generators to find the limits of the production setting

Problem

Jev can only choose from what reaches it, so evidence missing from the candidate pool is unreachable. I needed to know how far the production candidate generator preserved the answer-bearing evidence.

Experiment and result

Reusing stored scores, I regenerated rankings for four candidate generators and compared how well each preserved the 97 pre-registered citation groups across the 80 answerable questions: BGE-M3 alone, the production hybrid (0.75 embedding + 0.25 normalized BM25), RRF, and an alternating union.

At 100 candidates, the questions that preserved every citation were 74/80 for BGE-M3 alone, 63/80 for the production hybrid, 63/80 for RRF and 65/80 for the alternating union. MRR@500 over each citation's best rank was 0.1811 for BGE-M3 alone against 0.1245 for the production hybrid, with median ranks of 19 and 40. Beyond 300 candidates the four converged to 79/80 and 78/80.

Decision

The production hybrid weighting was ranking answer-bearing evidence lower than plain BGE-M3. That said, this is a ranking replay over stored scores on an evaluation set I had already reused, so I do not treat it as independent verification, and at the current 300-candidate setting the gap is small enough that I did not change production. I recorded it as a precondition: cutting the candidate count further requires fixing this ranking gap first.



Questions with sufficient evidence: 53/80 → 73/80; total median time 2.251s

Evidence coverage (LLM evaluation on golden set)
66.25% → 91.25%
 Original selector 53/80 → tuned Jev 73/80

Jev evidence-selection time
Median 9.090s → 2.251s
 500 candidates, sequential → 300 candidates, 4 parallel batches

False evidence returns on 20 unanswerable questions
20 → 0
 Original selector → tuned Jev

On a self-authored golden set of 80 answerable questions, the share with evidence supporting every required fact rose from 66.25% to 91.25%. On a separate 20 unanswerable questions, false evidence returns fell from 20 to 0. The improved selector was still slower than the original, but I chose Jev because accuracy mattered more to the service. I also tried grouping equivalent expressions: correct answers stayed similar on 76 questions (71 previously, 72 with Jev), while median decision time fell from 8.162s to 0.254s.

Scope and tradeoffs

The speedup compares Jev before and after its own settings changed. Retrieval gathers candidates and Jev makes the final evidence call, so the selection stage can be swapped on its own. Jev is an external decision model released in September 2026, so I have no long-run operational evidence for it yet, and even after tuning it stays slower than the previous selector. The candidate-generator comparison is a ranking replay over stored scores, and it measures candidate preservation rather than final answer accuracy.

[Introducing Jev · TypeSafe AI ↗](#)

CASE 02

Fruition / AI Agent · User-defined Skills · HITL · Prompt injection

User-defined Agent Skills with controlled execution

Built Agent Skills so users could define and reuse recurring document tasks, such as meeting notes. Connected instruction drafting and checks to user review and publication, followed by separate approval for actual changes.

2026.08 - 2026.09

LangGraph

Agent Skill

Prompt injection defenses

LLM Evaluation

Human-in-the-loop

Context engineering

Rolling conversation memory

Background

Built a workflow from Skill authoring and publishing to actual document changes, allowing users to define and reuse their own recurring tasks. AI drafts the instructions and the edits, code checks permissions and execution scope, and the user decides what is published and applied.

Original Agent

94 / 114

Code changes, without evaluator

98 / 114

Final implementation with evaluator

104 / 114

Decisions and implementation

1 Authoring and publishing user-defined instructions

Problem

Recurring tasks such as meeting notes vary by user; developers cannot prebuild every workflow.

Decision

Let users define and reuse natural-language instructions instead of adding each workflow as a feature. Treat AI-generated instructions as proposals until users review them.

Implementation

Turned natural-language requests into reusable Markdown Skills. Users review and edit drafts; the Skill and version are stored on final publication.

Verification

Neither Skills nor temporary drafts persist in the DB before publication. Published instructions can be reused in later tasks.

2 Checking instructions for permission and approval bypasses

Problem

User instructions and reference documents could contain personal data or directions to bypass permissions and approval. Such intent can be phrased many ways, and users can edit drafts after review.

Decision

Use code for explicit risk and permission checks, and an LLM to detect rephrased bypass intent. Do not allow publication based only on an earlier check after users edit the content.

Implementation

Applied rule checks to input and reference documents. The server verified that risky phrases reported by the LLM existed in the source. After checking access and original documents, it sent only necessary structure to the model. Publication reran checks on the final content. Risky phrases were not silently deleted; only user-requested regeneration removed them and rechecked the result.

Verification

Showed blocked input and the problematic location to users, letting them edit it themselves or request regeneration.

Decisions and implementation (continued)

3 Previewing changes and applying them after approval

Problem

Publishing a reviewed Skill does not approve every later document change. Users still need to inspect the target documents and proposed modifications.

Decision

Separate instruction publication from approval of actual changes. Keep task classification in the router and execution in the executor, while letting users inspect what will change.

Implementation

Displayed before/after diffs for edits and a folder-and-document move plan for organization. Code checked output format and edit scope; only approved changes were executed.

Verification

With 6 documents, the plan–approve–execute check made no changes before approval. After approval, it created 3 topic folders and moved the documents.

Decisions and implementation (continued)

4 Deciding the scope of conversation context and edit input

Problem

A long conversation cannot be passed to the model in full, and sending a whole document for an edit drives up cost and latency. I had to decide what to keep and what to cut.

Decision

Keep the last six turns verbatim and compress everything older into a rolling summary. For edits, send only the target line range plus surrounding context, and refuse a range that straddles the boundary of a table or code block.

Implementation

Once recent messages reach six, the rolling summary is regenerated from the previous summary plus those messages; if the summarization call fails the previous summary is kept so the conversation does not break. The router receives the summary and recent messages along with the routing decisions from earlier turns, so it can resolve what a referring expression points back to. Edit input is cut to the target line range with context lines above and below, and ranges crossing a structural boundary are rejected.

Verification

A benchmark measures, across document lengths, how large the transmitted input is relative to the source and the average and p95 processing time.



Reusable tasks and approval workflow; editing passes 94/114 → 104/114

Original Agent

94 / 114

82.5%

Code changes, without evaluator

98 / 114

86.0%

Final implementation with evaluator

104 / 114

91.2%

Users can define recurring tasks, publish them as Skills and reuse them. The final instructions are rechecked on publication; actual document changes require a preview and user approval. In a plan–approve–execute check using 6 documents, folder changes occurred only after approval. Editing quality was evaluated separately on the same 114 drafts from 19 request types. The original implementation passed 94; improved edit-goal and retry inputs passed 98; adding semantic evaluation and rewriting passed 104. Evaluation checked request fulfillment and meaning preservation. Rewriting based on failure reasons was limited to one attempt, and a second failure returned no edit proposal. AI drafts instructions and edits, code checks permissions and execution scope, and users decide publication and final application.

Scope and tradeoffs

Normalized average editing-evaluation time increased from 11.11s to 18.92s. I accepted the latency while limiting rewrites.

[Building user-defined Agent Skills \(Korean\)](#) ↗

[The two sides of LLM evaluation for reliability \(Korean\)](#) ↗

CASE 03

Fruition / PDF conversion · Data pipeline

PDF → Markdown: internal pass rate 45.17% → 89.89%

To use research papers as knowledge, PDF-to-Markdown conversion had to preserve table values and equations. I traced omissions introduced during AI repair and split processing paths by content type and original layout.

2026.07 — 2026.09

Open-source PDF conversion: AnyDoc, Docling

LLM API

Python

LLM-as-a-Judge

Background

Using research papers as service knowledge required Markdown that preserved not only body text but also tables, equations and figures.

Existing Docling

45.17%

Improved pipeline

89.89%

Missing regions

4 → 1

Experiments and improvements

1 Selective repair and missed errors

Problem

Open-source PDF-to-Markdown conversion damaged table row/column relationships and equations. I needed a way to repair errors without processing everything again.

Experiment and result

Selected damaged regions for AI repair, then evaluated repaired and untouched regions separately. Of 96 repaired regions, 85 passed, but 107 errors or omissions remained in untouched content. A separate experiment adding OCR comparison raised the pass rate from 80.00% to 97.53%, but also selected 50 correct regions for repair; the sum of processing-stage times was about 12 minutes 35 seconds.

Decision

Selecting repair targets missed errors, and extra checks to catch them added cost. I decided to test whether having AI read every original page could avoid this dependence on damage detection.

2 Table/equation errors and cost in full-page AI conversion

Problem

Selective repair missed errors. Adding OCR checks also flagged correct content, increasing evaluation time and cost.

Experiment and result

Full AI conversion of 25 research PDFs (415 pages) achieved a 91.92% content-preservation pass rate. However, only 58.84% of tables and equations fully matched in values and structure, and processing every page incurred substantial API usage.

Decision

Rather than send all content through the same AI path, I decided to use AI only where needed for body text and give tables and equations inputs that preserved their structure. I looked for fast text extraction with selective AI repair.

Experiments and improvements (continued)

3 Comparing text extraction and adopting AnyDoc

Problem

Full-page AI processing still left table/equation errors and required sending all body text to the model. I needed fast text extraction alongside separate table/equation repair.

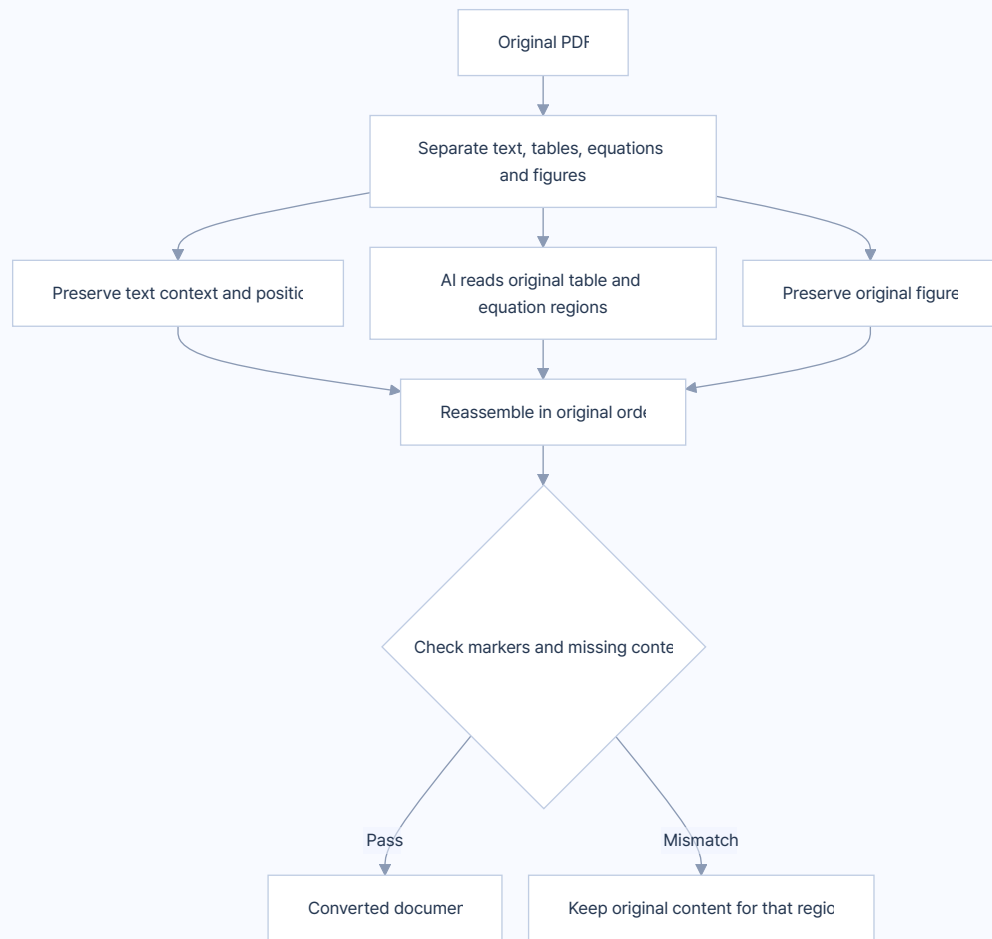
Experiment and result

I found AnyDoc on Threads and, after seeing its GitHub comparison with Docling, tested it on the same 4 PDFs (30 pages).

The recorded Docling run took 134.19s, while AnyDoc text extraction took 1.68s, about 80 times shorter. Docling's time also included OCR and table analysis. One AnyDoc document had garbled characters because the PDF lacked mappings from character codes to actual characters.

Decision

The processing scopes differed, so this was not an 80-fold improvement in complete conversion. Still, AnyDoc was useful for fast text extraction. I kept separate table/equation repair, switched body extraction to AnyDoc, used AI on garbled or misordered pages, and preserved original figures.



Pass rate against 30 original pages: 45.17% → 89.89%

Existing Docling

45.17%

30 pages total

Improved pipeline

89.89%

30 pages total

Missing regions

4 → 1

30 pages total

Split the same papers into 445 evaluation units covering body text, tables, equations and other content, then compared them with the originals. The existing Docling output and improved pipeline output were evaluated with method names hidden.

Scope and tradeoffs

Internal LLM evaluation of 4 PDFs (30 pages). Minor differences were allowed for text, headings and figures; tables and equations had to match in values, symbols and structure.

[Trying a small LLM for PDF → Markdown \(Korean\) ↗](#)

[Trying the open-source PDF converter AnyDoc \(Korean\) ↗](#)

CASE 04

Fruition / Kafka asynchronous processing · AI pipeline

Parallel document analysis: 282.11s → 73.89s

Sequential Wiki generation could make large documents slow to process and hurt the user experience. I measured worker scaling and resource use, then set different scaling policies for document analysis and embedding generation.

2026.09

AI pipeline

Kafka

PostgreSQL

Concurrency and memory

Background

I tested parallelism separately for document analysis and embedding generation to reduce waiting in the document-to-Wiki pipeline.

1 worker

282.11s

4 workers

73.89s

Combined process
memory

About 4×

Experiments and improvements

1 Worker count: analysis time versus memory

Problem

Sequential processing left later documents waiting for earlier external AI responses. I needed to measure both speed gains and memory cost from adding workers.

Experiment and result

Increasing workers from 1 to 4 for the same 4 documents reduced analysis time from 282.11s to 73.89s. Combined process memory rose from about 352MiB to 1,401MiB. I also checked that writes to the same workspace waited on a lock while different workspaces proceeded in parallel.

Decision

Processing another document during API waits helped, but resources grew with worker count. I decided to evaluate document workers separately from concurrent requests within each worker, and tested whether more concurrent requests could help further.

2 Extra concurrent requests and GPU contention

Problem

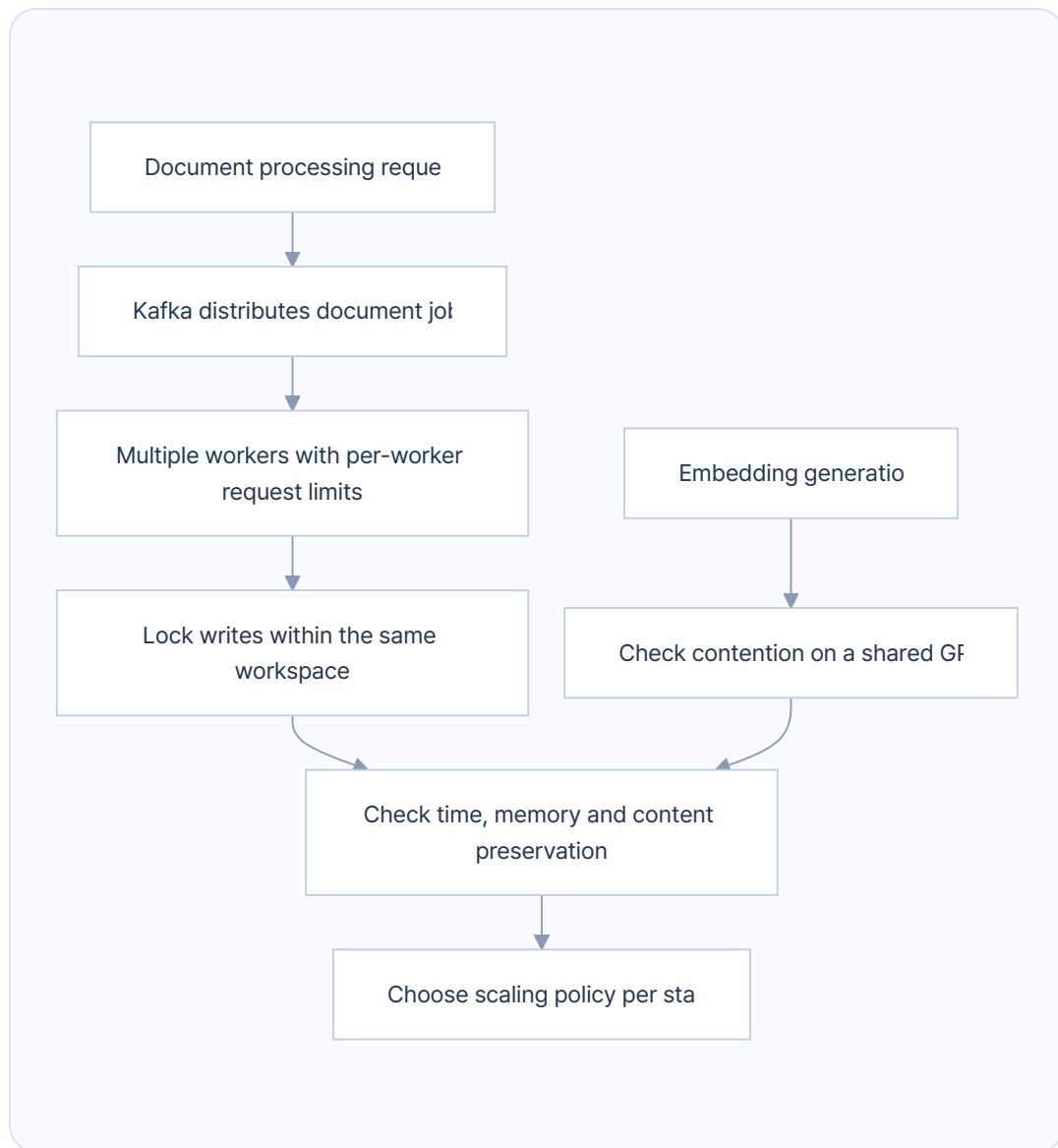
More workers reduced analysis time, but it still felt slow. It was unclear whether more requests per worker or more embedding processes on the same GPU would help.

Experiment and result

With 4 workers, raising concurrent AI requests per worker from 2 to 4 increased time from 73.89s to 95.92s. Running 2 embedding processes on the same GPU also raised each process's time from about 9.5s to 19–21s.

Decision

Analysis waiting on external responses and embeddings sharing a device behaved differently. Higher concurrency did not always improve speed, so I rejected these changes.



73.8% less analysis time for 4 documents · 300 queued jobs with zero loss

1 worker

282.11s

4 documents, 8 calls

4 workers

73.89s

Up to 8 concurrent API requests

Combined process memory

About 4x

351.7 → 1,401.0MiB

With 4 workers and 2 concurrent requests each, analysis time for 4 documents fell by about 73.8%. Memory rose about fourfold, and output quality varied with configuration. Replaying saved AI responses for 100 documents under 3 queue configurations processed all 300 jobs without omissions or failures. I adopted a maximum of 4 workers: although memory increased about fourfold, I judged that the AI service's MSA deployment on EKS could distribute the load.

Scope and tradeoffs

One analysis run per configuration on the same documents. These are not end-to-end timings including storage and post-processing.

PROJECT OVERVIEW

Pilltip

A personalized AI medication service that explains drug information in plain language and helps users manage their own and their family’s medications and schedules.

How it started

Started as a Pusan National University capstone project. Drug information is scattered, so it is hard to check whether a given medicine is safe for your current health condition. We set out to make that easy.

[GitHub repository ↗](#)

Team and my role

MY ROLE

Backend & AI

Drug and supplement DB, search and autocomplete, reminders, risk and caution information, data preparation, AI chatbot

- 01

Backend and AI development
- 02

Drug and supplement modeling, search, autocomplete and risk information
- 03

Medication reminders and logs
- 04

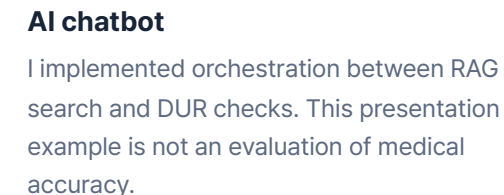
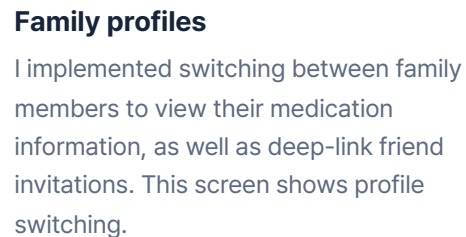
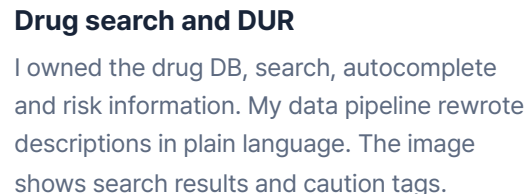
Friends and family
- 05

Data pipeline: \$200 estimated → \$11.18 spent, about 94% lower
- 06

RAG-based AI chatbot
- 07

Extended support to health functional foods after exhibition feedback

Android, 1	The Kotlin-based Android app
Backend & security, 1	User DB, login, questionnaires, AES-GCM encryption
Design	Designer



Drug data pipeline: \$200 estimated → \$11.18 spent

Built a data preparation pipeline to rewrite difficult drug information in plain language. Merged exact duplicates and then equivalent sentences, filled the missing 5% using ingredient-specific precautions, and converted unique text once before mapping it back to each drug. Actual API spending was \$11.18.

2025.07 — 2025.07

Data pipeline

Fuzzy Matching

GPT Batch API

Data analysis

Background

Built a feature to rewrite about 44,000 drug records into explanations users could understand.

Estimated cost for
original text

**About
\$200**

Actual API spending

\$11.18

Input after
preprocessing

326MB

Problem analysis and implementation

1 Removing exact duplicate sentences

Problem

Converting about 44,000 records totaling 1GB was estimated to cost about \$200 in API usage. I needed to cover all drugs within budget.

Experiment and result

Examined 100 records and found about 70% duplicate content. Used this sample and a work plan to secure 7 days for preprocessing. Split text into sentence blocks, merged exact matches and preserved links from each drug to its blocks.

Decision

Convert repeated sentences once and reuse them rather than cover fewer drugs. Since non-identical duplicates remained, the next step was to identify equivalent sentences with different wording.

2 Merging semantically equivalent sentences

Problem

Exact deduplication left sentences that differed only in endings or phrasing. I needed to avoid repeated conversion while preserving dosage and numeric differences.

Experiment and result

Normalized sentence endings and compared similarity to group equivalent sentences. Preserved drug-to-block mappings, handled dosages and numbers separately, and manually checked samples. Exact and semantic deduplication reduced input from 1GB to 326MB.

Decision

Reuse equivalent explanations while preserving drug-specific facts and numbers, rather than merge solely on string similarity. Next I checked missing information instead of stopping at deduplication.

Problem analysis and implementation (continued)

3 Filling the missing 5% with ingredient-specific precautions

Problem

About 5% of information was missing after earlier processing, leaving gaps in descriptions for the full target set.

Experiment and result

Inspected missing entries and noticed that drugs with the same ingredient shared precautions. Collected and cleaned data for the missing ingredients, then processed it to fill the gaps.

Decision

Fill gaps by ingredient instead of excluding records. Shared ingredient precautions maintained coverage without repeatedly processing each drug.

4 GPT Batch API conversion and mapping results back to drugs

Problem

Deduplication reduced input, but I still needed to minimize model costs.

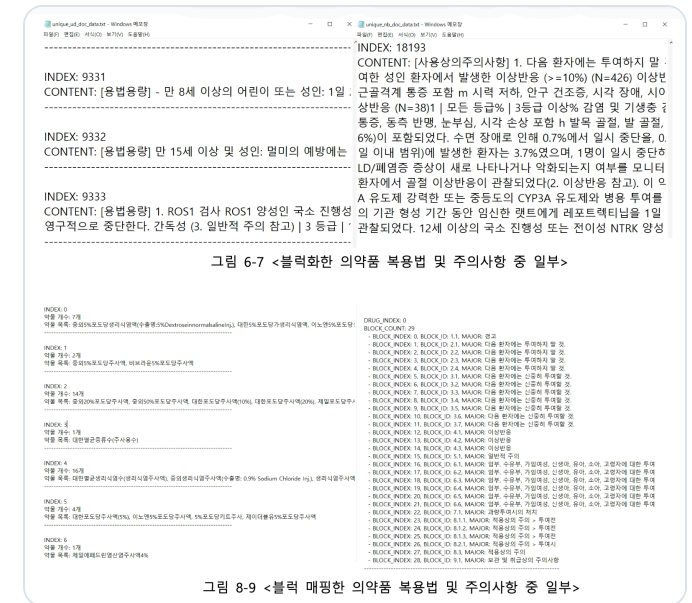
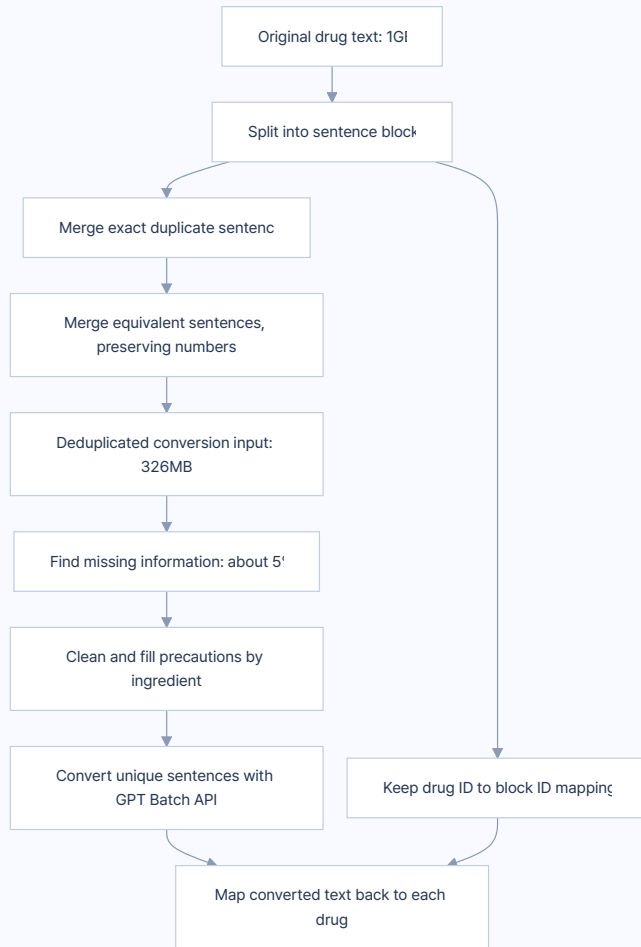
Experiment and result

The work did not require immediate responses during service use, so batch waiting was acceptable. Converted unique sentences with GPT Batch API, then reused the preserved drug-to-block mappings to attach results to each drug. Actual API spending was \$11.18 versus about \$200 estimated for all original text.

Decision

Combined deduplication, ingredient-level gap filling and batch conversion to deliver the feature without reducing drug coverage.

Improved processing flow



Removed exact and semantic duplicates separately, filled missing information by ingredient, and mapped converted text back to each drug.

Conversion input: 1GB → 326MB; actual API spend: \$11.18

Completed conversion for \$11.18 in actual API spending versus about \$200 estimated for all original text. Removed exact and semantic duplicates, then filled the missing 5% with cleaned ingredient-specific precautions. Reusing converted unique sentences kept the feature within budget without reducing the set of drugs covered.

Scope and tradeoffs

About \$200 was the upfront estimate; \$11.18 was actual spending. Labor for 7 days of preprocessing is excluded.

Estimated cost for original text

About \$200

Estimate

Actual API spending

\$11.18

Conversion work

Input after preprocessing

326MB

Original: 1GB

CASE 06

Pilltip / AI Agent · Sensitive data boundaries

Medication chatbot: keeping health profiles out of external AI inputs

Built a medication chatbot using RAG and AI orchestration for app integration. Separated symptom-based drug discovery from health-profile decisions, using the model for language understanding and code to control sensitive profile use.

2025.09

Spring AI

Weaviate

RAG

AI Orchestration

Background

Built a chatbot that finds drug information from natural-language symptoms and personalizes it using health profiles.

Design and implementation

1 Semantic search connecting symptoms to drug indications

Problem

Everyday symptom descriptions and drug indications use different words for the same meaning. Related information needed to be found without exact keyword matches.

Decision

Use semantic search to connect symptoms and indications, with separate processing paths for the information each question needs.

Implementation

Stored drug indications in Weaviate for semantic symptom search. Built paths for drug discovery, medication risks and dosage information according to the question.

Verification

Implemented natural-language symptom search and routing to the appropriate information-processing path.

2 Internal health-profile processing, separate from external AI inputs

Problem

Personalization required pregnancy status, medications and underlying conditions, but sending those profiles to an external LLM risked exposing sensitive information.

Decision

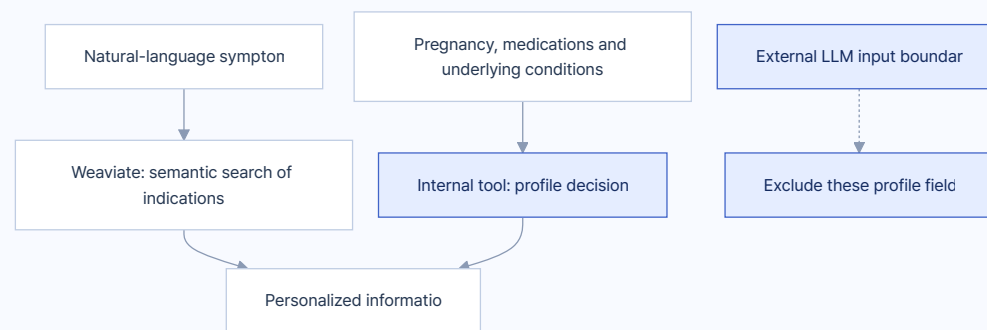
Separate language understanding from profile-condition decisions. Keep profile decisions in internal code where inputs and rules can be controlled directly.

Implementation

Excluded pregnancy, medication and underlying-condition fields from external LLM inputs. Connected internal profile decisions to symptom search results to provide personalized information.

Verification

Separated processing paths so those health-profile fields stayed out of external LLM inputs while remaining available for internal personalization.



Separating symptom search from internal health-profile decisions

Enabled natural-language drug discovery while excluding pregnancy status, current medications and underlying conditions from external LLM inputs. Explicitly separated information sent to AI from information evaluated by internal code. In a domain that handles sensitive data, I settled the boundary between what the model may judge and what code stays responsible for before writing the implementation.

Scope and tradeoffs

Profile-rule changes require code updates. I prioritized keeping those personal data fields out of external AI inputs and retained profile decisions in internal code.

Beyond project development

Rhwp · Fixing HWPX save errors

PR #1213 merged to fix textFlow attribute preservation. Contributed issue analysis, implementation, tests and CI fixes.

[GitHub repository](#) ↗

APPTIVE · Backend mentoring

Used my experience as a mentee to improve teaching. Led 6 mentoring and code-review sessions for 12 mentees.

945-day Baekjoon problem-solving streak

Aiming for one problem a day, I reached a 945-day streak on solved.ac. 1,659 problems solved, solved.ac Platinum IV.

[Solutions repository](#) ↗

Project reflections

ARCHITECTURE / Data ownership

Moving to MSA and learning from feedback

Shifted from splitting by feature to defining data ownership. Documented both the benefits of asynchronous processing and the added operational burden.

PLANNING / Defining user problems

Reflections on design thinking

Revisited feature-first planning and brought together the team's difficulties finding documents to define the problem we would solve together.

[More blog posts ↗](#)

Jaehyeong Kim

AI Application Developer · Backend Engineer

kkuldangi2@gmail.com

martinel2.github.io

github.com/Martinel2

I owned the whole AI side of an LLM workspace, designing and building its RAG, agent and evaluation layers.

I weigh accuracy, cost and response time against each other, confirm the trade-off with an evaluation, and then choose the implementation that fits the service.